1 Introduction

Table des matières

1 Introduction **1**

2 Stratégie de validation **2**

3 Infrastructure de test **2**

4 Gestion des risques et des rendus **2**

5 Résultats de couverture (Jacoco) **3**

6 Autres méthodes de validation **3**

2 Stratégie de validation

3 Infrastructure de test

4 Gestion des risques et des rendus

Afin d'évaluer et de prévenir au mieux les risques du projet, nous avons identifié les dangers susceptibles d'affecter la qualité ou le bon fonctionnement du projet. Ces risques ont été classés par ordre décroissant selon leur rapport coût-bénéfice (du plus critique au moins critique), conformément aux recommandations des enseignants.

4.1 Analyse des dangers critiques

Pour chaque danger identifié, nous avons défini une ou plusieurs actions facilement vérifiables, permettant d'éviter ce risque ou d'en limiter l'impact. Ces dangers sont, à ce stade, centrés sur le sous-langage «sans objet», mais s'appliquent également à la version complète du langage ainsi qu'au développement de l'extension. Une réévaluation des risques sera donc réalisée après le rendu intermédiaire.

- **Validité fonctionnelle** : Pour le rendu intermédiaire, il est impératif que l'intégralité de la chaîne de compilation du sous-langage «sans objet» soit fonctionnelle (conception, vérifications contextuelles et génération de code).
  ⇒ À partir de l'exemple fourni dans le polycopié, nous avons identifié les points critiques, notamment l'ajout de la classe `ConvFloat` à l'arbre abstrait lors de certaines opérations arithmétiques ou d'assignations, ou encore l'appel à `ReadInt`. Ces points ont été ajoutés aux tests unitaires de l'étape d'analyse contextuelle en plus des règles données par la spécification du langage Deca.
- **Conformité de l'interface et automatisation** : Le non-respect des spécifications de la commande `decac` (options `-p`, `-v`, `-r` etc.) ou une sortie interactive lors de `mvn test` rendrait le compilateur incompatible avec les scripts d'évaluation automatique des enseignants.
  ⇒ Nous avons mis en place un script de test d'interface dédié vérifiant le code de retour (*exit status* 0 ou 1) ainsi que le format de sortie pour chaque option. Les tests exécutés automatiquement par `Maven` sont strictement non-interactifs. Les programmes nécessitant une interaction utilisateur (`readInt`, `readFloat` par exemple) ou les scripts utilisés lors des démos en séance de suivi, sont isolés dans des sous-répertoires `/interactive` et exclus de l'exécution automatique.
- **Validité des oracles** : Il existe un risque que nos tests passent alors que le compilateur produit des résultats incorrects (acceptation de programmes invalides, rejet de programmes valides, AST incohérent ou exceptions Java non-rattrapées).
  ⇒ Pour y pallier nous utilisons des oracles automatiques, c'est-à-dire des critères qui déterminent automatiquement si un test a réussi ou échoué. Par exemple la vérification de l'idempotence de la décompilation, la cohérence des numéros de lignes d'erreur ou encore l'absence d'exceptions Java non-rattrapées. Ces oracles sont inspirés des exemples fournis dans le polycopié.
- **Erreurs de régression** : Le développement de la génération de code pourrait, à travers certaines erreurs, rendre le compilateur invalide sur des étapes précédentes pourtant fonctionnelles.
  ⇒ Intégration du script `common-tests.sh` dans notre cycle de développement. L'exécution de la commande `mvn clean test` doit être réalisée avant chaque *push* sur le dépôt afin de garantir que les nouvelles fonctionnalités ne cassent pas les fonctionnalités déjà implémentées.
- **Risque de synchronisation Git** : L'oubli d'un fichier source ou l'ajout accidentel de fichiers binaires/auxiliaires (`.class`, etc.) pourrait créer des conflits ou empêcher la compilation correcte sur une autre machine.
  ⇒ Nous avons séparé les fichiers générés dans des répertoires spécifiques (`target/` et `/docs/build`) pour minimiser les erreurs d'oubli dans le `.gitignore`. Avant chaque rendu, un `git clone` est réalisé dans un répertoire vierge pour vérifier que la compilation (`mvn test-compile`) et les tests (`mvn test`) fonctionnent sans dépendances locales (cf. 4.3)

4.2 Gestion des rendus et calendrier

La réussite du projet repose aussi sur la gestion du calendrier et des livrables. Nous avons identifié deux dangers organisationnels majeurs : le dépassement des échéances et la livraison d'une version instable du projet.

- **Respect des échéances** : L'oubli d'une date de rendu, de suivi, de soutenance ainsi qu'un des documents qui nous sont demandés pour ces écheances n'est pas souhaitable.
  ⇒ Nous avons ajouté à notre planning des jalons anticipés, fixés 24 heures avant chaque rendu officiel, passé lesquels aucune nouvelle fonctionnalité ne doit être ajoutée. Ces deux jours sont réservés exclusivement à la vérification et la validation de rendu. Le planning partagé récapitule les dates clés du projet ainsi que les jalons internes.
- **Gestion des versions** : Le danger est de livrer une version instable suite à l'intégration de dernière minute d'une fonctionnalité complexe.
  ⇒ Pour le rendu intermédiaire, la priorité est donnée au compilateur «sans objet» sur la branche `main`. Toute autre fonctionnalité (extensions, partie objet) est développée sur des branches séparées et ne sera fusionnée qu'après validation.

4.3 Procédure de validation de rendu

Pour reproduire notre validation, voici la liste des tâches exécutées avant chaque rendu.

- ☐ Lancer `mvn test` pour exécuter l'ensemble des tests (unitaires et fonctionnels). Cette commande exécute aussi le script `common-tests.sh` : aucune erreur ne doit survenir.
- ☐ Vérifier que les tests interactifs (utilisant `readInt` ou `readFloat`) sont tous situés dans des sous-répertoires `/interactive`.
- ☐ Vérifier le format des en-têtes `// Description` et `// Resultats` de chaque fichier de test `.deca` et s'assurer que la ligne d'erreur attendue pour les tests `invalid` correspond à la sortie du compilateur.
- ☐ Exécuter la procédure de simulation de rendu ci-dessous.

Simulation de rendu : L'usage du répertoire `/tmp/` garantit que ce dernier sera nettoyé au redémarrage de l'ordinateur et évite ainsi de polluer les dossiers de travail personnels avec des versions dupliquées du projet, mais le choix d'un autre dossier de destination n'importe pas.

1. Clonage du dépôt dans un répertoire temporaire :

```
$ git clone https://gitlab.ensimag.fr/gl2026/gr6/gl36.git /tmp/Projet_GL_Rendu
  cd /tmp/Projet_GL_Rendu
```

2. Vérification de la compilation et des tests :

```
$ mvn test
```

3. Vérification de l'exécutable :

```
$ ./bin/decac -p src/test/deca/syntax/valid/hello.deca
```

5 Résultats de couverture (Jacoco)

6 Autres méthodes de validation